

BANCO DE DADOS – SQL 3

Profa. Vandecia Fernandes – BiCT/ECP/UFMA

Referências: Elmasri, R. and Navathe, S.B. Sistemas de Bancos de Dados. Korth, H.F. e Silberschatz, A. Sistemas de Bancos de Dados. Agradecimentos aos professores: Simara Rocha, Claudio Baptista e Eduardo Viana.



VISÕES



VISÕES

- Uma visão é uma tabela virtual que é definida a partir de outras tabelas, contendo sempre os dados atualizados.
- É uma relação que não armazena dados, composta dinamicamente por uma consulta que é previamente analisada e otimizada.
- É um meio de prover ao usuário um “modelo personalizado” do banco de dados

VISÕES

- A visão é definida por uma DDL e é computada cada vez que são realizadas consultas aos dados daquela visão.
- Um SGBD pode dar suporte a um grande número de visões sobre qualquer conjunto de relações
- Dois objetivos principais das visões:
 - Simplificar consultas
 - Autorização de acesso (segurança)

VISÕES

- **Sintaxe de Criação:**

- `CREATE VIEW nomeVisão AS expressão_de_consulta`

- **Exemplo:**

```
CREATE VIEW vAltoEscalao AS  
  SELECT codEmp, nome, salario  
  FROM empregado (Tabela existente no BD)  
  WHERE salario > 10000
```

VISÕES

- **Visão com várias tabelas:**

Projeto (**codProj**, tipo, descricao)

ProjetoEmpregado (**codProj**, **codEmp**, dataInicial, dataFinal)

Empregado (**codEmp**, nome, categoria, salario)

```
CREATE VIEW vExemploVisao AS
```

```
SELECT e.codEmp, e.nome, e.salario, p.descricao
```

```
FROM empregado e, projeto p, projetoEmpregado pe
```

```
WHERE e.salario > 10000 AND e.codEmp = pe.codEmp AND  
pe.codProj = p.codProj
```

VISÕES

- Exemplo 2: Criar uma visão que contém informações gerenciais sobre um departamento, contendo o nome do depto, total de empregados e total de salários.
- ```
CREATE VIEW InfoDepto
AS SELECT D.nome, COUNT(*), SUM(salario)
FROM Departamento d, Empregado e
WHERE d.coddep = e.depto GROUP BY d.nome
```

# VISÕES

- Consultas SQL podem ser especificadas sobre a visão .
- Toda visão pode ser consultada mas nem toda visão pode ser atualizada.
- Exemplo:  
SELECT nome  
FROM vProjetosAltoEscalao (visão existente no BD)  
WHERE descricao = “Projeto A”



# VISÕES

- **Sintaxe de Eliminação:**  
DROP VIEW nomeVisão
- Exemplo:  
DROP VIEW exemploVisao

# VISÕES

- **Atualização de Visões:**

- Visões são úteis em consultas, mas existem restrições em relação a atualizações.

- **Observações:**

- Uma visão definida numa única tabela é atualizável se os atributos da visão contém a chave primária.
- Visões definidas sobre múltiplas tabelas usando junção geralmente não são atualizáveis.
- Visões usando funções de agrupamento e agregados não são atualizáveis.



# LINGUAGEM PROCEDURAL

# LINGUAGEM PROCEDURAL

- Os SGDB disponibilizam recursos para manipulação de dados por meio de PL (linguagens procedurais). A PL permite que a lógica do negócio fique armazenada no banco de dados e seja executada a partir de comandos SQL.
- A programação em PL é feita por meio de function(função), Store Procedure (procedimentos armazenados) e Trigger (gatilho). Cada SGDB tem sua maneira própria para tratar esses elementos de programação.

## LINGUAGEM PROCEDURAL

- Diferentemente de outros SGBDs que tratam os conceitos de procedimentos armazenados, gatilhos e funções como coisas distintas, o PostgreSQL trata todos eles como funções.
- Essas estruturas tem características diferentes, mas são todas criadas como funções(functions).

## LINGUAGEM PROCEDURAL

- O PostgreSQL aceita por padrão as linguagens PL/PgSQL, SQL, PL/Perl, PL/Python e PL/TCL.
- Outras linguagens podem ser adicionadas como extensão, mas a linguagem de procedimentos PL/PgSQL é a mais utilizada



# LINGUAGEM PROCEDURAL – FUNÇÕES

# FUNÇÃO

- Sintaxe de Criação:

CREATE [ OR REPLACE ] FUNCTION  
nome\_funcao (variáveis e tipos)

[ RETURNS tipo ]

<corpo da funcao>

LANGUAGE linguagem utilizada



# FUNÇÃO

- Para usar a função:

`SELECT nome_função (parâmetros)`

- Sintaxe de exclusão:

`DROP FUNCTION nome_função (tipo  
parametros)`

# FUNÇÃO

## ■ Exemplo 1:

```
CREATE OR REPLACE FUNCTION kmtom(double precision) RETURNS
double precision AS
'select $1 * 1000'
language SQL;
```

# FUNÇÃO

## ■ Exemplo 2:

CREATE OR REPLACE FUNCTION cal\_idade(date) RETURNS DOUBLE  
PRECISION AS

```
'select extract(year from age(current_date,$1)) '
language sql;
```

# FUNÇÃO

## ■ Exemplo 3:

```
CREATE OR REPLACE FUNCTION CLASSIFICA(AGE) RETURNS TEXT AS
'SELECT CASE
WHEN CAL_IDADE($1)>=0 AND CAL_IDADE($1)<15 THEN "CRIANÇA"
WHEN CAL_IDADE($1)>=15 AND CAL_IDADE($1)<=18 THEN "ADOLESCENTE"
WHEN CAL_IDADE($1)>18 AND CAL_IDADE($1)<30 THEN "JOVEM ADULTO"
WHEN CAL_IDADE($1)>=30 THEN "ADULTO"
END'
LANGUAGE SQL;
```

# FUNÇÃO

- Exemplo 4: Criar uma função para somar 2 valores

```
CREATE OR REPLACE FUNCTION somar(um integer, dois integer)
 RETURNS integer AS $$

 BEGIN

 RETURN um + dois;

 END;

$$ LANGUAGE plpgsql;
```



# LINGUAGEM PROCEDURAL – TRIGGER

# TRIGGER

- Uma trigger (gatilho) é uma ação que o SGBD deve desencadear **antes** ou **após** os comandos INSERT, UPDATE OU DELETE
- A trigger está associada a uma função: primeiro criamos uma função para depois criarmos a trigger. A função de gatilho deve ser declarada como uma função que não recebe argumentos

# TRIGGER

- Tempo
  - BEFORE
  - AFTER
- Evento
  - INSERT
  - UPDATE
  - DELETE



# TRIGGER

- Sintaxe de Criação:

```
CREATE TRIGGER nome_trigger { BEFORE | AFTER } { event }
 ON nome_tabela [FOR [EACH] { ROW | STATEMENT }]
 EXECUTE PROCEDURE funcao_criada ()
```

# TRIGGER

- FOR EACH ROW:
  - o procedimento em execute procedure é chamado uma vez para cada linha que a operacao modifica.
- FOR EACH STATEMENT:
  - ocorrerá somente uma chamada ao procedimento descrito em execute procedure, nao importando quantas linhas sejam modificadas .

# TRIGGER

- Exemplo 1: Executa a função check\_account sempre que uma linha da tabela accounts for atualizada.

```
CREATE TRIGGER check_update
 BEFORE UPDATE ON accounts
 FOR EACH ROW
 EXECUTE PROCEDURE check_account();
```

# TRIGGER

## ■ Exemplo 2: Passo 1: Criar a Função

```
CREATE or replace FUNCTION atualiza_cada() RETURNS trigger AS $atualiza$
BEGIN
 IF NEW.nome IS NULL THEN
 RAISE EXCEPTION 'O nome nao pode ser nulo';
 END IF;

 NEW.data_cadastro := current_timestamp;
 RETURN NEW;
END;
$atualiza$ LANGUAGE plpgsql;
```

# TRIGGER

- Exemplo2: Passo 2 - Criar a Trigger

```
CREATE TRIGGER atualiza BEFORE INSERT OR UPDATE ON sql3.pessoa
FOR EACH ROW EXECUTE PROCEDURE atualiza_cada();
```

# TRIGGER

- Sintaxe de Exclusão:

```
DROP TRIGGER nome_trigger ON nome_tabela;
```



DÚVIDAS?